FeedOps — Master Project Development Document

Project: FeedOps — Feed Management Hub
Team: Rushik, Dhruv, Rushit
Target launch: Monday, 19 October 2026
Development approach: One shared GitHub repository, three developers, coordinated direct pushes to main.

1. Project objective

Build FeedOps from scratch as a unified feed management and intelligence platform, based on the provided 33-page project reference PDF.

The platform aims to collect, organize, search, analyze and monitor information from multiple sources through one interface.

The PDF describes RSS/Atom feeds, news APIs, article analysis, historical tracking, translation, search, collaboration, weather information, cybersecurity intelligence, financial information, podcasts, radio, aviation feeds and geospatial mapping.

The first release must prioritize a reliable, working core product. Advanced modules should be added only when feasible and tested.

2. Required core functionality

1. Dashboard showing collected articles and feed status.
2. Add, manage, refresh and remove RSS/Atom feeds.
3. Fetch and store articles with normalized metadata.
4. Display article lists and individual article details.
5. Search and filter articles by title, source, category and date where available.
6. AI-assisted article summaries and categorization, if the selected AI service is available.
7. Basic error handling, logging and empty/error states.
8. Responsive user interface.
9. Configuration through environment variables.
10. Setup instructions and deployment documentation.

Do not claim that a feature is complete until it works in the integrated application.

3. Technology rules

- Backend: Python and Flask.
- Frontend: HTML, CSS and vanilla JavaScript unless the team explicitly agrees otherwise.

- Database: SQLite for a local MVP, using feed.db in the instance directory. Use a common database abstraction so PostgreSQL can be considered for production.
- Feed ingestion: a suitable RSS/Atom parsing library.
- AI: an agreed external API or model, configured through environment variables.
- Version control: one shared GitHub repository and one main branch.
- Testing: Python tests for backend logic and manual integration tests for the user interface.

Keep the implementation straightforward, modular and maintainable. Do not introduce unnecessary frameworks or services close to launch.

4. Team responsibilities and file ownership

Rushit — Project foundation and feed ingestion

Primary responsibility:

- Application entry point and Flask initialization.
- Configuration and shared environment-variable conventions.
- Database connection helper and schema.
- RSS/Atom fetching and normalization.
- Feed CRUD and refresh functionality.
- Integration of common backend interfaces.
- Final integration and launch checks.

Owned files:

- run.py
- config.py
- app/__init__.py
- app/database/db.py
- app/database/schema.sql
- app/routes/feeds.py
- app/services/feed_service.py
- Related feed-ingestion tests.

Rushik — Article processing, AI and search

Primary responsibility:

- Article processing and AI analysis.
- Article summaries and categorization.
- Sentiment and relevance scoring where feasible.
- Search service and filtering logic.
- Article listing and detail functionality.
- Related tests and documentation.

Owned files:

- app/services/ai_service.py
- app/services/search_service.py
- app/routes/articles.py
- app/routes/search.py
- app/templates/articles.html
- app/templates/article_detail.html
- Related CSS, JavaScript and tests.

Dhruv — Dashboard and frontend integration

Primary responsibility:

- Dashboard and navigation.
- Feed management interface.
- Article cards and reusable UI components.
- Dashboard statistics and visualizations.
- Responsive layout, loading states and error states.
- Frontend integration testing.

Owned files:

- app/routes/dashboard.py
- app/templates/base.html
- app/templates/dashboard.html
- app/templates/feeds.html
- app/static/css/
- app/static/js/

- Related frontend tests and documentation.

Each person may contribute to frontend and backend development, but must coordinate before changing another person's owned files.

5. Shared project structure

FeedOps/

- run.py
- config.py
- requirements.txt
- .env.example
- .gitignore
- README.md
- CONTRIBUTING.md
- app/
    - **init**.py
    - routes/
    - services/
    - database/
    - templates/
    - static/
- tests/
- instance/
    - feed.db (local runtime file, excluded from Git)

Create only necessary files. Avoid committing empty placeholder files that do not serve a purpose.

6. Shared article data contract

Every stored article should use consistent field names:

- id: internal primary key.
- feed_id: reference to its source feed.
- title: article title.
- url: canonical article URL.

- summary: source-provided or extracted summary.
- content: extracted text when available.
- author: author when available.
- published_at: publication timestamp when available.
- fetched_at: timestamp when the application retrieved the article.
- category: optional category.
- sentiment: optional AI-generated sentiment.
- ai_summary: optional AI-generated summary.
- relevance_score: optional score.
- content_hash: optional field for duplicate detection.

The database schema and Python models must agree on these fields. Optional fields may be empty when information is unavailable.

A feed should have a stable ID, title, URL, site name, enabled status, last-fetch time and error status.

7. Shared function contracts

Feed ingestion:

fetch_feed(url) returns normalized article data and handles invalid URLs, parsing failures and network errors.

Article processing:

analyze_article(article) returns available analysis fields, such as category, summary, sentiment and relevance score.

Search:

search_articles(query, filters) returns matching articles using the agreed database and filtering rules.

Routes must return consistent JSON responses for API endpoints and appropriate HTML pages for browser routes. Agree on actual route paths before implementing dependent modules.

8. Shared configuration

Use a local .env file for secrets and local settings. Commit .env.example, never the real .env.

Expected variables may include:

- SECRET_KEY
- AI_API_KEY
- AI_MODEL

- DATABASE_PATH

Do not hardcode credentials. Do not share personal GitHub access tokens. Each developer must authenticate using their own authorized GitHub account.

9. Git collaboration rules

1. Everyone clones the same repository.
2. Before beginning a task, run git status and fetch/pull the latest main.
3. Work only on assigned files unless the owner agrees to changes.
4. Commit small, meaningful changes.
5. Push to main only after coordinating with teammates.
6. Never force-push to main.
7. If a push is rejected, fetch and reconcile remote changes before retrying.
8. Never overwrite another developer's changes to resolve a conflict without reviewing them.
9. Run the application and relevant tests after integrating changes.
10. Never commit .env, API keys, database files containing private data, or other secrets.

For shared files, coordinate before editing. Direct pushes do not eliminate the need for conflict resolution.

10. Launch priorities

P0 — Required:

- Application starts successfully.
- Database initializes correctly.
- Feed ingestion works.
- Articles are stored and displayed.
- Search and basic filtering work.
- Main pages are responsive and usable.
- Configuration and deployment instructions are correct.

P1 — Include if reliable:

- AI summaries.
- Automatic categories.
- Dashboard analytics.

- Duplicate detection.
- Basic notifications.

P2 — Future development:

- Full translation suite.
- Complete article-version history.
- Team collaboration and advanced permissions.
- Weather and market intelligence.
- Podcast and radio directories.
- Live aviation audio and advanced geospatial tools.
- The remaining specialized modules described in the reference PDF.

11. Definition of done

A feature is complete only when:

- Its assigned implementation exists.
- It uses the agreed shared interfaces.
- It handles expected errors.
- It does not break another module.
- Relevant tests pass.
- It has been tested in the integrated application.
- Its status is accurately documented.

12. Important rule

All three developers and their AI assistants must follow this document. If a requirement is missing or two modules disagree, stop and coordinate rather than silently inventing a different interface or modifying shared files.